SuperfluidGPU on bwUniCluster 3.0

Code changes and how to run the code on CPUs and GPUs

6 October 2026 · repository `superfluid-gpu` (changes relative to commit `f3ede5a`) and `super-analysis/run_dcc`

Overview

The simulation now builds and runs on the H100 nodes of bwUniCluster 3.0, gives the same results on CPU and GPU, has a new measurement of the 3D power spectra of the fields, and runs a time step about twice as fast on one GPU as at the start.

Change	Files	Effect
1. 3D FFT engine	`measurer.h`	GPU builds no longer allocate FFTW host arrays
2. Topological charge readout	`measurer.h/.cxx`	no N^3 arrays per rank, no host round trips
3. Field spectra (new)	`measurer*`, `ModelA.h`, `SuperPions.cxx`	new output `fields_Sk`
4. Diffusion stencil	`Stepper.cxx`	kernel 3.58 → 2.54 ms
5. Single-GPU ghost fill	`Stepper.cxx`, `Profiling.h`	ghost exchange 4.71 → 1.71 ms
6. Job scripts	`sjg_runner.py`, `sconfigurator_jg.py`	CPU and GPU jobs on bwUniCluster

Time per step for a 256^3 lattice on one H100 (step sequence `ABBBBBB`, no measurements, `-log_view`):

Version	Ghost exchange per call	Stencil kernel per call	Step	Speedup
At the start	4.71 ms	3.58 ms	57.4 ms	1.0×
After change 4 (stencil)	4.71 ms	2.54 ms	50.5 ms	1.14×
After change 5 (ghost fill)	1.71 ms	2.44 ms	29.7 ms	1.93×

Kernel times come from Nsight Compute (stencil) and Nsight Systems (ghost exchange). Each step does 7 ghost exchanges (one per substep) and 6 stencil passes. For comparison, a full 128-core Perlmutter CPU node needed 280 ms per 256^3 step (`docs/PLAN.md`), so one H100 is now about 9× faster than a CPU node.

Correctness. After changes 4 and 5 the fields after 20 steps are bit-identical to the original code. CPU and GPU runs with the same seed and the same number of ranks agree to rounding level: fields to $\leq 4 \cdot 10^{-14}$ after 40 steps, and both spectra to $\leq 10^{-13}$, also against an independent numpy recomputation from field snapshots (Section 1.7).

Contents

1 Code changes **3**

1.1 3D FFT engine `measurer_fft_3d` (`measurer.h`) . . . 3

1.2 Topological charge readout (`measurer.h`, `measurer.cxx`) . . . 5

1.3 New measurement: 3D power spectra of the fields . . . 12

1.4 Diffusion stencil (`Stepper.cxx`) . . . 14

1.5 Single-GPU ghost fill (`Stepper.cxx`, `Profiling.h`) . . . 15

1.6 Job scripts (`super-analysis/run_dcc`) . . . 17

1.7 Validation . . . 20

2 Running the code **21**

2.1 The run modes . . . 21

2.2 One-time setup: PETSc . . . 21

2.3 Compiling SuperPions . . . 21

2.3.1 CPU build . . . 21

2.3.2 GPU build . . . 21

2.4 Starting jobs . . . 22

2.4.1 With the configurator (recommended) . . . 22

2.4.2 By hand: CPU job . . . 22

2.4.3 By hand: one GPU . . . 22

2.4.4 By hand: four GPUs on one node . . . 23

2.5 Options and input parameters . . . 23

2.6 Notes for multi-GPU runs . . . 23

1 Code changes

Each subsection shows the purpose of the change, the code that was removed, the code that was added, and how the change affects performance. Changes that were made and later replaced again (an intermediate compile fix of the binning kernel, test variants of the stencil, a first version of the ghost fill) are not listed; only the difference between the starting version and the current version is shown.

1.1 3D FFT engine `measurer_fft_3d` (`measurer.h`)

Purpose. The class computes the real-to-complex 3D FFT used for the topological charge spectrum and now also for the field spectra. It is used only on rank 0.

What changed.

- The old class always created an FFTW plan with `FFTW_MEASURE` and allocated an N^3 input array and an $N^2(N/2+1)$ complex output array on the host, even in GPU builds where cuFFT did the work. In GPU builds it also allocated a separate N^3 device input buffer, and every call copied the input host→device and the result device→host.
- The new class creates plans and buffers on first use. `execute_host` transforms host memory with FFTW. `execute_device` (GPU builds) transforms a device pointer directly with cuFFT and leaves the result on the device, so the input can be a PETSc Kokkos vector and needs no extra buffer.
- The transform is now unnormalized; the caller applies $1/N^3$ while binning, which saves one pass over the spectrum.

Performance. At $N = 1024$ the old class allocated about 17 GB of host memory and 8.6 GB of extra device memory on rank 0 and planned an FFTW transform at start-up even when no spectrum was measured. A GPU build now allocates only the cuFFT output and workspace, and only when a spectrum is measured.

Removed code (starting version, `measurer.h`):

```
// Class to compute a real-to-complex 3D fourier transform of a scalar field
// on a periodic box.  Normalization convention (same as measurer_fft, in 3D):
//
//   W(k) = (1 / (n0*n1*n2)) \sum_x e^{-i k.x} W(x)
//
// The dimensions are (n0, n1, n2) in row-major (C) order, so that the last
// index n2 is contiguous in memory.  The real input has size n0*n1*n2 and the
// complex output (r2c half spectrum) has size n0*n1*(n2/2+1).
//
// When built with SFGPU_CUFFT (Kokkos + CUDA), the FFT runs on the GPU via
// cuFFT and is typically 10-30x faster than single-rank FFTW for large grids.
// The FFTW path is kept as fallback for CPU-only builds.
class measurer_fft_3d {

private:
  size_t n0, n1, n2;
  size_t out_size;
#ifdef SFGPU_CUFFT
  cufftHandle cufft_plan;
  double *d_in = nullptr;
  cufftDoubleComplex *d_out = nullptr;
  bool use_cufft = true;
#endif
  // FFTW fallback (always available)
  std::vector<double> in_ptr;
  std::vector<std::complex<double>> out_ptr;
  fftw_plan fftw_plan_handle;

public:
  measurer_fft_3d(const size_t N0, const size_t N1, const size_t N2)
      : n0(N0), n1(N1), n2(N2),
        out_size(N0 * N1 * (N2 / 2 + 1)),
        in_ptr(N0 * N1 * N2, 0.),
        out_ptr(N0 * N1 * (N2 / 2 + 1), 0.) {
    // Always create the FFTW plan (cheap, and needed for non-CUDA builds)
    fftw_plan_handle = fftw_plan_dft_r2c_3d(
        static_cast<int>(N0), static_cast<int>(N1), static_cast<int>(N2),
        in_ptr.data(), reinterpret_cast<fftw_complex *>(out_ptr.data()),
        FFTW_MEASURE);
#ifdef SFGPU_CUFFT
    // Create the cuFFT plan and allocate device buffers
    if (cufftPlan3d(&cufft_plan, static_cast<int>(N0), static_cast<int>(N1),
                    static_cast<int>(N2), CUFFT_D2Z) != CUFFT_SUCCESS) {
      use_cufft = false;
    } else {
      cudaMalloc(&d_in, N0 * N1 * N2 * sizeof(double));
```

```
      cudaMalloc(&d_out, out_size * sizeof(cufftDoubleComplex));
      if (!d_in || !d_out) {
        // Allocation failed, fall back to FFTW
        if (d_in) cudaFree(d_in);
        if (d_out) cudaFree(d_out);
        d_in = nullptr;
        d_out = nullptr;
        cufftDestroy(cufft_plan);
        use_cufft = false;
      }
    }
#endif
  }

  ~measurer_fft_3d() {
    fftw_destroy_plan(fftw_plan_handle);
#ifdef SFGPU_CUFFT
    if (use_cufft) {
      cufftDestroy(cufft_plan);
      cudaFree(d_in);
      cudaFree(d_out);
    }
#endif
  }

  // Transform the real volume `in` (size n0*n1*n2) into the complex half
  // spectrum `out` (size n0*n1*(n2/2+1)), applying the 1/(n0*n1*n2) norm.
  void execute(const std::vector<double> &in,
               std::vector<std::complex<double>> &out) {
    const double norm = static_cast<double>(n0 * n1 * n2);
#ifdef SFGPU_CUFFT
    if (use_cufft) {
      const size_t in_size = n0 * n1 * n2;
      // Host -> device
      cudaMemcpy(d_in, in.data(), in_size * sizeof(double),
                 cudaMemcpyHostToDevice);
      // Execute R2C transform on GPU
      cufftExecD2Z(cufft_plan, d_in, d_out);
      cudaDeviceSynchronize();
      // Device -> host (cufftDoubleComplex is layout-compatible with
      // std::complex<double>)
      cudaMemcpy(out.data(), d_out, out_size * sizeof(std::complex<double>),
                 cudaMemcpyDeviceToHost);
      // Normalize
      for (size_t i = 0; i < out_size; i++) {
        out[i] /= norm;
      }
      return;
    }
#endif
    // FFTW fallback
    std::copy(in.begin(), in.end(), in_ptr.begin());
    fftw_execute(fftw_plan_handle);
    for (size_t i = 0; i < out_ptr.size(); i++) {
      out[i] = out_ptr[i] / norm;
    }
  }
};
```

Added code (current version):

```
// Class to compute a real-to-complex 3D fourier transform of a scalar field
// on a periodic box.  The transform is unnormalized,
//
//   W(k) = \sum_x e^{-i k.x} W(x),
//
// and callers divide by n0*n1*n2 (see Measurer::radialSpectrum).  The
// dimensions are (n0, n1, n2) in row-major (C) order, so that the last index
// n2 is contiguous in memory.  The real input has size n0*n1*n2 and the
// complex output (r2c half spectrum) has size n0*n1*(n2/2+1).
//
// execute_host transforms host memory with FFTW. With SFGPU_CUFFT,
// execute_device transforms device memory with cuFFT and leaves the result on
// the device. The plans and buffers are made on first use, so a GPU run never
// allocates the FFTW arrays and vice versa.
class measurer_fft_3d {

private:
  size_t n0, n1, n2;
  size_t out_size;
  // FFTW (host)
  std::vector<double> in_ptr;
  std::vector<std::complex<double>> out_ptr;
  fftw_plan fftw_plan_handle = nullptr;
#ifdef SFGPU_CUFFT
  // cuFFT (device)
  cufftHandle cufft_plan;
  bool have_cufft = false;
```

```
  cufftDoubleComplex *d_out = nullptr;
#endif

public:
  measurer_fft_3d(const size_t N0, const size_t N1, const size_t N2)
      : n0(N0), n1(N1), n2(N2), out_size(N0 * N1 * (N2 / 2 + 1)) {}

  ~measurer_fft_3d() {
    if (fftw_plan_handle) {
      fftw_destroy_plan(fftw_plan_handle);
    }
#ifdef SFGPU_CUFFT
    if (have_cufft) {
      cufftDestroy(cufft_plan);
      cudaFree(d_out);
    }
#endif
  }

  size_t size_in() const { return n0 * n1 * n2; }
  size_t size_out() const { return out_size; }

  // Transform the real volume `in` (host memory, size n0*n1*n2). Returns the
  // half spectrum in host memory, valid until the next call.
  const std::complex<double> *execute_host(const double *in) {
    if (!fftw_plan_handle) {
      in_ptr.assign(size_in(), 0.);
      out_ptr.assign(out_size, 0.);
      fftw_plan_handle = fftw_plan_dft_r2c_3d(
          static_cast<int>(n0), static_cast<int>(n1), static_cast<int>(n2),
          in_ptr.data(), reinterpret_cast<fftw_complex *>(out_ptr.data()),
          FFTW_MEASURE);
    }
    std::copy(in, in + size_in(), in_ptr.begin());
    fftw_execute(fftw_plan_handle);
    return out_ptr.data();
  }

#ifdef SFGPU_CUFFT
  // Transform the real volume `d_in` (device memory, size n0*n1*n2; not
  // modified). Returns the half spectrum in device memory, valid until the
  // next call.
  const cufftDoubleComplex *execute_device(const double *d_in) {
    if (!have_cufft) {
      if (cufftPlan3d(&cufft_plan, static_cast<int>(n0), static_cast<int>(n1),
                      static_cast<int>(n2), CUFFT_D2Z) != CUFFT_SUCCESS) {
        throw std::runtime_error("measurer_fft_3d: cufftPlan3d failed");
      }
      if (cudaMalloc(&d_out, out_size * sizeof(cufftDoubleComplex)) !=
          cudaSuccess) {
        cufftDestroy(cufft_plan);
        throw std::runtime_error("measurer_fft_3d: cudaMalloc failed");
      }
      have_cufft = true;
    }
    // An out-of-place D2Z transform leaves its input unchanged
    cufftExecD2Z(cufft_plan, const_cast<double *>(d_in), d_out);
    cudaDeviceSynchronize();
    return d_out;
  }
#endif
};
```

1.2 Topological charge readout (`measurer.h`, `measurer.cxx`)

Purpose. `computeTopChargeFourier` computes the topological charge density $q(x) = \frac{1}{12\pi^2}\,\epsilon^{abcd}\phi_a\,\partial_x\phi_b\,\partial_y\phi_c\,\partial_z\phi_d$ with centered differences, its 3D FFT, and the spectrum $S_q(k) = \langle|q(k)|^2\rangle$ averaged over radial shells $|k| \approx m \cdot 2\pi/L$, together with the zero mode $q(k=0)$. Output: `topcharge_Sk`, `topcharge_zeromode`, `topcharge_kbins`, `topcharge_Nshell` (unchanged names and normalization).

How it worked before. Every rank allocated a full N^3 array on the host (and in GPU runs another one on the device), wrote q at its own sites and zeros elsewhere, and `MPI_Reduce` summed all these arrays onto rank 0. In GPU runs the data then went device→host (q), host→device (cuFFT input), device→host (spectrum) and host→device again ($|q(k)|^2$ for the binning kernel). The GPU build did not compile at all: the binning kernel was a device lambda inside the private member function, which nvcc rejects (*"The enclosing parent function for an extended ___host___ ___device___ lambda cannot have private or protected access"*).

How it works now.

- `setupSpectra` creates, on first use, a 1-value-per-site copy of the lattice layout

(`DMDACreateCompatibleDMDA`, same domain decomposition and vector type, so Kokkos vectors stay on the GPU), a global vector `qglob`, and for several ranks a natural-order vector and a scatter to rank 0.
- The q kernel writes only the rank's own sites into `qglob`.
- `radialSpectrum` brings `qglob` to rank 0 in natural order with PETSc (`DMDAGlobalToNatural` + `VecScatterCreateToZero`); with one rank this step is skipped because the global order already is the natural order. On rank 0 it runs cuFFT directly on the GPU data (or FFTW for host data) and bins $|q(k)|^2$ on the GPU, so only the $\approx\sqrt{3}N/2$ shell values come back to the host.
- All kernels are free functions, which fixes the nvcc compile error.

Performance and limits.

- Memory: each rank holds only its own block of q instead of N^3 on host and device. Rank 0 holds the full q once plus the FFT output (about 17 GB at 1024^3).
- Communication: the full grid crosses the network once (each rank sends its block) instead of every rank sending N^3 values in `MPI_Reduce`.
- Host transfers: on one GPU no N^3 array is copied between host and device any more (before: four copies per measurement).
- The old `MPI_Reduce` used an `int` count and would have failed above $N \approx 1290$; PETSc uses 64-bit sizes.
- The time per measurement before and after was not benchmarked.

Removed code (starting version, `measurer.cxx`: q kernel and `computeTopChargeFourier`):

```
#ifdef SFGPU_KOKKOS
// GPU kernel: compute q(x) at each owned site and write into a device buffer
// indexed in natural (lexicographic) order: idx = i + N*j + N*N*k.
// Sites not owned by this rank stay zero so MPI_Reduce(SUM) assembles all.
static PetscErrorCode topcharge_kernel_kokkos(
    DM da, Vec localU, PetscInt N, double prefac,
    double inv2hx, double inv2hy, double inv2hz,
    Kokkos::View<double *> &qdev) {
  PetscFunctionBeginUser;
  PetscInt xs, ys, zs, xm, ym, zm;
  PetscCall(DMDAGetCorners(da, &xs, &ys, &zs, &xm, &ym, &zm));

  ConstNodeView u;
  PetscCall(DMDAVecGetKokkosOffsetViewDOF(da, localU, &u));

  const PetscInt Nlat = N;
  Kokkos::parallel_for(
      "TopChargeDensity",
      SitePolicy({zs, ys, xs}, {zs + zm, ys + ym, xs + xm}),
      KOKKOS_LAMBDA(PetscInt k, PetscInt j, PetscInt i) {
        constexpr int Nphi = ModelAData::Nphi;
        double phi[Nphi], Dx[Nphi], Dy[Nphi], Dz[Nphi];
        for (int a = 0; a < Nphi; a++) {
          phi[a] = u(k, j, i, a);
          Dx[a] = (u(k, j, i + 1, a) - u(k, j, i - 1, a)) * inv2hx;
          Dy[a] = (u(k, j + 1, i, a) - u(k, j - 1, i, a)) * inv2hy;
          Dz[a] = (u(k + 1, j, i, a) - u(k - 1, j, i, a)) * inv2hz;
        }
        const double q = prefac * det4x4(phi, Dx, Dy, Dz);
        const size_t idx = static_cast<size_t>(i) +
                           static_cast<size_t>(Nlat) * j +
                           static_cast<size_t>(Nlat) * Nlat * k;
        qdev(idx) = q;
      });
  Kokkos::fence();

  PetscCall(DMDAVecRestoreKokkosOffsetViewDOF(da, localU, &u));
  PetscFunctionReturn(PETSC_SUCCESS);
}
#endif

void Measurer::computeTopChargeFourier(Vec *solution) {
  DM &da = model->domain;
  Vec localU;
  DMGetLocalVector(da, &localU);
  ghost_exchange(da, *solution, localU);

  const auto &data = model->data;
  const double hx = data.hX();
  const double hy = data.hY();
  const double hz = data.hZ();
  const double prefac = 1.0 / (12.0 * M_PI * M_PI);

  PetscInt ixs, iys, izs, nx, ny, nz;
  DMDAGetCorners(da, &ixs, &iys, &izs, &nx, &ny, &nz);

  const size_t Ntot = static_cast<size_t>(N) * N * N;
  std::vector<double> qlocal(Ntot, 0.0);
```

```
#ifdef SFGPU_KOKKOS
  if (is_kokkos_vec(*solution)) {
    // GPU path: compute q(x) on device, copy to host
    Kokkos::View<double *> qdev("topcharge_q", Ntot);
    Kokkos::deep_copy(qdev, 0.0);
    topcharge_kernel_kokkos(da, localU, N, prefac,
                            1.0 / (2.0 * hx), 1.0 / (2.0 * hy),
                            1.0 / (2.0 * hz), qdev);
    auto qhost = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), qdev);
    std::memcpy(qlocal.data(), qhost.data(), Ntot * sizeof(double));
  } else
#endif
  {
    // Host path
    G_node ***fld;
    DMDAVecGetArrayRead(da, localU, &fld);

    for (int k = izs; k < izs + nz; k++) {
      for (int j = iys; j < iys + ny; j++) {
        for (int i = ixs; i < ixs + nx; i++) {
          double phi[ModelAData::Nphi], Dx[ModelAData::Nphi],
              Dy[ModelAData::Nphi], Dz[ModelAData::Nphi];
          for (int a = 0; a < ModelAData::Nphi; a++) {
            phi[a] = fld[k][j][i].f[a];
            Dx[a] = (fld[k][j][i + 1].f[a] - fld[k][j][i - 1].f[a]) / (2.0 * hx);
            Dy[a] = (fld[k][j + 1][i].f[a] - fld[k][j - 1][i].f[a]) / (2.0 * hy);
            Dz[a] = (fld[k + 1][j][i].f[a] - fld[k - 1][j][i].f[a]) / (2.0 * hz);
          }
          const double q = prefac * det4x4(phi, Dx, Dy, Dz);
          const size_t idx = static_cast<size_t>(i) +
                             static_cast<size_t>(N) * j +
                             static_cast<size_t>(N) * N * k;
          qlocal[idx] = q;
        }
      }
    }

    DMDAVecRestoreArrayRead(da, localU, &fld);
  }

  DMRestoreLocalVector(da, &localU);

  // Assemble the full q(x) volume on rank 0.
  int rank = -1;
  MPI_Comm_rank(PETSC_COMM_WORLD, &rank);
  std::vector<double> qfull;
  if (rank == 0) {
    qfull.assign(Ntot, 0.0);
  }
  MPI_Reduce(qlocal.data(), rank == 0 ? qfull.data() : nullptr,
             static_cast<int>(Ntot), MPI_DOUBLE, MPI_SUM, 0, PETSC_COMM_WORLD);

  if (rank != 0) {
    return;
  }

  // 3D FFT.  The FFTW/cuFFT dimensions are (n0,n1,n2) = (NZ,NY,NX) so that
  // the x direction is the contiguous (last) index, matching the natural
  // ordering idx = i + N*j + N*N*k used to fill qfull above.
  const size_t nxh = static_cast<size_t>(N / 2 + 1);
  std::vector<std::complex<double>> qtilde(static_cast<size_t>(N) * N * nxh);
  fftw3d->execute(qfull, qtilde);

  // Zero mode / DC component.
  topcharge_zero = qtilde[0];

  // Spherical (azimuthal) average of the power spectrum |qtilde(k)|^2.
  // Modes are binned by |k| into shells of width dk = 2*pi/L.
  std::fill(topcharge_Sk.begin(), topcharge_Sk.end(), 0.0);
  std::fill(topcharge_Nshell.begin(), topcharge_Nshell.end(), 0.0);

#ifdef SFGPU_CUFFT
  // GPU path: run the spherical binning on device with atomic adds.
  // The qtilde array is already on the host after the cuFFT execute+copy.
  // We upload it to the device, bin there, and download the small result.
  {
    const int Nlat = N;
    const int nbins = NtopchargeBins;
    const size_t nk = static_cast<size_t>(N) * N * nxh;

    // Upload qtilde as pairs of doubles (re, im packed contiguously)
    Kokkos::View<double *> qt_dev("qtilde_norm2", nk);
    {
      // Pre-compute |qtilde|^2 on host and upload (cheaper than uploading
      // complex + computing norm on device for this one-shot measurement)
      std::vector<double> norm2(nk);
      for (size_t i = 0; i < nk; i++) norm2[i] = std::norm(qtilde[i]);
      auto qt_host = Kokkos::View<double *, Kokkos::HostSpace,
```

```
                                Kokkos::MemoryUnmanaged>(norm2.data(), nk);
      Kokkos::deep_copy(qt_dev, qt_host);
    }

    Kokkos::View<double *> Sk_dev("Sk", nbins);
    Kokkos::View<double *> Ns_dev("Nshell", nbins);
    Kokkos::deep_copy(Sk_dev, 0.0);
    Kokkos::deep_copy(Ns_dev, 0.0);

    const int half = Nlat / 2;
    const size_t nxh_local = nxh;
    Kokkos::parallel_for(
        "TopchargeBinning",
        Kokkos::MDRangePolicy<Kokkos::Rank<3>>(
            {0, 0, 0}, {Nlat, Nlat, static_cast<int>(nxh_local)}),
        KOKKOS_LAMBDA(int kz, int ky, int kx) {
          const int mz = (kz <= half) ? kz : kz - Nlat;
          const int my = (ky <= half) ? ky : ky - Nlat;
          const int mx = kx;
          const double r = Kokkos::sqrt(
              static_cast<double>(mx * mx + my * my + mz * mz));
          int m = static_cast<int>(Kokkos::floor(r + 0.5));
          if (m >= nbins) m = nbins - 1;
          const size_t idx =
              (static_cast<size_t>(kz) * Nlat + static_cast<size_t>(ky)) *
                  nxh_local +
              static_cast<size_t>(kx);
          Kokkos::atomic_add(&Sk_dev(m), qt_dev(idx));
          Kokkos::atomic_add(&Ns_dev(m), 1.0);
        });
    Kokkos::fence();

    // Download small result arrays
    auto Sk_host = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), Sk_dev);
    auto Ns_host = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), Ns_dev);
    for (int m = 0; m < nbins; m++) {
      topcharge_Nshell[m] = Ns_host(m);
      topcharge_Sk[m] = (Ns_host(m) > 0.0) ? Sk_host(m) / Ns_host(m) : 0.0;
    }
  }
#else
  // CPU path
  {
    const int half = N / 2;
    for (int kz = 0; kz < N; kz++) {
      const int mz = (kz <= half) ? kz : kz - N;
      for (int ky = 0; ky < N; ky++) {
        const int my = (ky <= half) ? ky : ky - N;
        for (int kx = 0; kx < static_cast<int>(nxh); kx++) {
          const int mx = kx;
          const double r =
              sqrt(static_cast<double>(mx * mx + my * my + mz * mz));
          int m = static_cast<int>(floor(r + 0.5));
          if (m >= NtopchargeBins) {
            m = NtopchargeBins - 1;
          }
          const size_t idx =
              (static_cast<size_t>(kz) * N + static_cast<size_t>(ky)) * nxh +
              static_cast<size_t>(kx);
          topcharge_Sk[m] += std::norm(qtilde[idx]);
          topcharge_Nshell[m] += 1.0;
        }
      }
    }
    for (int m = 0; m < NtopchargeBins; m++) {
      if (topcharge_Nshell[m] > 0.0) {
        topcharge_Sk[m] /= topcharge_Nshell[m];
      }
    }
  }
#endif
}
```

Added code (current version, `measurer.cxx`: shell index, kernels, binning, gather and FFT, `computeTopChargeFourier`):

```
// Radial shell m of the mode (kz, ky, kx) of the r2c half spectrum: modes
// with |k| = m*dk (rounded) share a shell, the last shell collects the corners
#ifdef SFGPU_KOKKOS
O4_FUNCTION
#else
static inline
#endif
int shell_index(int kz, int ky, int kx, int Nlat, int nbins) {
  const int half = Nlat / 2;
  const int mz = (kz <= half) ? kz : kz - Nlat;
  const int my = (ky <= half) ? ky : ky - Nlat;
  const double r = sqrt(static_cast<double>(kx * kx + my * my + mz * mz));
```

```
  const int m = static_cast<int>(floor(r + 0.5));
  return (m < nbins) ? m : nbins - 1;
}

#ifdef SFGPU_KOKKOS
// One value per owned site of the 1-dof DMDA used for the spectra
using SiteValueView =
    Kokkos::Experimental::OffsetView<PetscScalar ***, Kokkos::LayoutRight,
                                     KokkosMemSpace>;

// GPU kernel: compute q(x) at each owned site into q (a global vector of
// da1). localU is the ghosted solution.
static PetscErrorCode topcharge_kernel_kokkos(DM da, Vec localU, DM da1,
                                              Vec q, double prefac,
                                              double inv2hx, double inv2hy,
                                              double inv2hz) {
  PetscFunctionBeginUser;
  PetscInt xs, ys, zs, xm, ym, zm;
  PetscCall(DMDAGetCorners(da, &xs, &ys, &zs, &xm, &ym, &zm));

  ConstNodeView u;
  PetscCall(DMDAVecGetKokkosOffsetViewDOF(da, localU, &u));
  SiteValueView qv;
  PetscCall(DMDAVecGetKokkosOffsetViewWrite(da1, q, &qv));

  Kokkos::parallel_for(
      "TopChargeDensity",
      SitePolicy({zs, ys, xs}, {zs + zm, ys + ym, xs + xm}),
      KOKKOS_LAMBDA(PetscInt k, PetscInt j, PetscInt i) {
        constexpr int Nphi = ModelAData::Nphi;
        double phi[Nphi], Dx[Nphi], Dy[Nphi], Dz[Nphi];
        for (int a = 0; a < Nphi; a++) {
          phi[a] = u(k, j, i, a);
          Dx[a] = (u(k, j, i + 1, a) - u(k, j, i - 1, a)) * inv2hx;
          Dy[a] = (u(k, j + 1, i, a) - u(k, j - 1, i, a)) * inv2hy;
          Dz[a] = (u(k + 1, j, i, a) - u(k - 1, j, i, a)) * inv2hz;
        }
        qv(k, j, i) = prefac * det4x4(phi, Dx, Dy, Dz);
      });
  Kokkos::fence();

  PetscCall(DMDAVecRestoreKokkosOffsetViewWrite(da1, q, &qv));
  PetscCall(DMDAVecRestoreKokkosOffsetViewDOF(da, localU, &u));
  PetscFunctionReturn(PETSC_SUCCESS);
}

// GPU kernel: copy the field component c of the solution U into q (a global
// vector of da1)
static PetscErrorCode field_component_kokkos(DM da, Vec U, PetscInt c, DM da1,
                                             Vec q) {
  PetscFunctionBeginUser;
  PetscInt xs, ys, zs, xm, ym, zm;
  PetscCall(DMDAGetCorners(da, &xs, &ys, &zs, &xm, &ym, &zm));

  ConstNodeView u;
  PetscCall(DMDAVecGetKokkosOffsetViewDOF(da, U, &u));
  SiteValueView qv;
  PetscCall(DMDAVecGetKokkosOffsetViewWrite(da1, q, &qv));

  Kokkos::parallel_for(
      "FieldComponent",
      SitePolicy({zs, ys, xs}, {zs + zm, ys + ym, xs + xm}),
      KOKKOS_LAMBDA(PetscInt k, PetscInt j, PetscInt i) {
        qv(k, j, i) = u(k, j, i, c);
      });
  Kokkos::fence();

  PetscCall(DMDAVecRestoreKokkosOffsetViewWrite(da1, q, &qv));
  PetscCall(DMDAVecRestoreKokkosOffsetViewDOF(da, U, &u));
  PetscFunctionReturn(PETSC_SUCCESS);
}
#endif

#ifdef SFGPU_CUFFT
// GPU kernel: bin scale*|ft(k)|^2 of the half spectrum ft (device memory)
// into the radial shells with atomic adds. A free function because nvcc
// rejects device lambdas in private members.
static void radial_binning_kokkos(const cufftDoubleComplex *ft, int Nlat,
                                  int nbins, double scale,
                                  const Kokkos::View<double *> &Sk,
                                  const Kokkos::View<double *> &Ns) {
  const int nxh = Nlat / 2 + 1;
  Kokkos::parallel_for(
      "RadialBinning",
      Kokkos::MDRangePolicy<
          Kokkos::Rank<3, Kokkos::Iterate::Right, Kokkos::Iterate::Right>>(
          {0, 0, 0}, {Nlat, Nlat, nxh}),
      KOKKOS_LAMBDA(int kz, int ky, int kx) {
        const int m = shell_index(kz, ky, kx, Nlat, nbins);
```

```
        const size_t idx =
            (static_cast<size_t>(kz) * Nlat + static_cast<size_t>(ky)) * nxh +
            static_cast<size_t>(kx);
        const cufftDoubleComplex z = ft[idx];
        Kokkos::atomic_add(&Sk(m), scale * (z.x * z.x + z.y * z.y));
        Kokkos::atomic_add(&Ns(m), 1.0);
      });
  Kokkos::fence();
}
#endif

// Bin scale*|ft(k)|^2 of the half spectrum ft (host memory) into the radial
// shells
static void radial_binning_host(const std::complex<double> *ft, int Nlat,
                                int nbins, double scale, double *Sk,
                                double *Ns) {
  const int nxh = Nlat / 2 + 1;
  for (int kz = 0; kz < Nlat; kz++) {
    for (int ky = 0; ky < Nlat; ky++) {
      for (int kx = 0; kx < nxh; kx++) {
        const int m = shell_index(kz, ky, kx, Nlat, nbins);
        const size_t idx =
            (static_cast<size_t>(kz) * Nlat + static_cast<size_t>(ky)) * nxh +
            static_cast<size_t>(kx);
        Sk[m] += scale * std::norm(ft[idx]);
        Ns[m] += 1.0;
      }
    }
  }
}

PetscErrorCode Measurer::setupSpectra() {
  PetscFunctionBeginUser;
  if (da1) {
    PetscFunctionReturn(PETSC_SUCCESS);
  }
  // A 1-dof DMDA with the same layout as the domain, and the same vector
  // type (Kokkos vectors stay on the device)
  DM &da = model->domain;
  VecType vtype;
  PetscCall(DMDACreateCompatibleDMDA(da, 1, &da1));
  PetscCall(DMGetVecType(da, &vtype));
  PetscCall(DMSetVecType(da1, vtype));
  PetscCall(DMCreateGlobalVector(da1, &qglob));

  PetscMPIInt size, rank;
  PetscCallMPI(MPI_Comm_size(PETSC_COMM_WORLD, &size));
  PetscCallMPI(MPI_Comm_rank(PETSC_COMM_WORLD, &rank));
  if (size > 1) {
    PetscCall(DMDACreateNaturalVector(da1, &qnat));
    PetscCall(VecScatterCreateToZero(qnat, &toZero, &qseq));
  }
  if (rank == 0) {
    fftw3d = std::make_unique<measurer_fft_3d>(N, N, N);
  }
  PetscFunctionReturn(PETSC_SUCCESS);
}

PetscErrorCode Measurer::radialSpectrum(double *Sk, double *Nshell,
                                        std::complex<double> *zero) {
  PetscFunctionBeginUser;
  // Bring q to rank 0 in natural ordering, idx = i + N*j + N*N*k. With one
  // rank the global ordering of the DMDA already is the natural one.
  Vec qfull = qglob;
  if (toZero) {
    PetscCall(DMDAGlobalToNaturalBegin(da1, qglob, INSERT_VALUES, qnat));
    PetscCall(DMDAGlobalToNaturalEnd(da1, qglob, INSERT_VALUES, qnat));
    PetscCall(
        VecScatterBegin(toZero, qnat, qseq, INSERT_VALUES, SCATTER_FORWARD));
    PetscCall(
        VecScatterEnd(toZero, qnat, qseq, INSERT_VALUES, SCATTER_FORWARD));
    qfull = qseq;
  }
  PetscMPIInt rank;
  PetscCallMPI(MPI_Comm_rank(PETSC_COMM_WORLD, &rank));
  if (rank != 0) {
    PetscFunctionReturn(PETSC_SUCCESS);
  }

  // S(k) = <|q(k)|^2> per shell with q(k) = (1/N^3) \sum_x e^{-ikx} q(x)
  const int nbins = NtopchargeBins;
  const double ntot = static_cast<double>(fftw3d->size_in());
  const double scale = 1.0 / (ntot * ntot);
  std::fill(Sk, Sk + nbins, 0.0);
  std::fill(Nshell, Nshell + nbins, 0.0);

  const PetscScalar *q;
  PetscMemType mtype;
  PetscCall(VecGetArrayReadAndMemType(qfull, &q, &mtype));
```

```
#ifdef SFGPU_CUFFT
  if (PetscMemTypeDevice(mtype)) {
    // FFT and binning on the device; only the shells come back to the host
    Kokkos::fence();
    const cufftDoubleComplex *ft = fftw3d->execute_device(q);
    Kokkos::View<double *> Sk_dev("Sk", nbins);
    Kokkos::View<double *> Ns_dev("Nshell", nbins);
    radial_binning_kokkos(ft, N, nbins, scale, Sk_dev, Ns_dev);
    auto Sk_host =
        Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), Sk_dev);
    auto Ns_host =
        Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), Ns_dev);
    for (int m = 0; m < nbins; m++) {
      Sk[m] = Sk_host(m);
      Nshell[m] = Ns_host(m);
    }
    cufftDoubleComplex z0;
    cudaMemcpy(&z0, ft, sizeof(z0), cudaMemcpyDeviceToHost);
    *zero = std::complex<double>(z0.x, z0.y) / ntot;
  } else
#endif
  {
    (void)mtype;
    const std::complex<double> *ft = fftw3d->execute_host(q);
    radial_binning_host(ft, N, nbins, scale, Sk, Nshell);
    *zero = ft[0] / ntot;
  }
  PetscCall(VecRestoreArrayReadAndMemType(qfull, &q));

  for (int m = 0; m < nbins; m++) {
    if (Nshell[m] > 0.0) {
      Sk[m] /= Nshell[m];
    }
  }
  PetscFunctionReturn(PETSC_SUCCESS);
}

void Measurer::computeTopChargeFourier(Vec *solution) {
  setupSpectra();

  DM &da = model->domain;
  Vec localU;
  DMGetLocalVector(da, &localU);
  ghost_exchange(da, *solution, localU);

  const auto &data = model->data;
  const double inv2hx = 1.0 / (2.0 * data.hX());
  const double inv2hy = 1.0 / (2.0 * data.hY());
  const double inv2hz = 1.0 / (2.0 * data.hZ());
  const double prefac = 1.0 / (12.0 * M_PI * M_PI);

#ifdef SFGPU_KOKKOS
  if (is_kokkos_vec(*solution)) {
    topcharge_kernel_kokkos(da, localU, da1, qglob, prefac, inv2hx, inv2hy,
                            inv2hz);
  } else
#endif
  {
    PetscInt xs, ys, zs, xm, ym, zm;
    DMDAGetCorners(da, &xs, &ys, &zs, &xm, &ym, &zm);
    G_node ***fld;
    DMDAVecGetArrayRead(da, localU, &fld);
    PetscScalar ***q;
    DMDAVecGetArray(da1, qglob, &q);
    for (PetscInt k = zs; k < zs + zm; k++) {
      for (PetscInt j = ys; j < ys + ym; j++) {
        for (PetscInt i = xs; i < xs + xm; i++) {
          double phi[ModelAData::Nphi], Dx[ModelAData::Nphi],
              Dy[ModelAData::Nphi], Dz[ModelAData::Nphi];
          for (int a = 0; a < ModelAData::Nphi; a++) {
            phi[a] = fld[k][j][i].f[a];
            Dx[a] = (fld[k][j][i + 1].f[a] - fld[k][j][i - 1].f[a]) * inv2hx;
            Dy[a] = (fld[k][j + 1][i].f[a] - fld[k][j - 1][i].f[a]) * inv2hy;
            Dz[a] = (fld[k + 1][j][i].f[a] - fld[k - 1][j][i].f[a]) * inv2hz;
          }
          q[k][j][i] = prefac * det4x4(phi, Dx, Dy, Dz);
        }
      }
    }
    DMDAVecRestoreArray(da1, qglob, &q);
    DMDAVecRestoreArrayRead(da, localU, &fld);
  }
  DMRestoreLocalVector(da, &localU);

  radialSpectrum(topcharge_Sk.data(), topcharge_Nshell.data(),
                 &topcharge_zero);
}
```

Changes in the class `Measurer` (`measurer.h`; red removed, green added). These include the members for the field spectra of Section 1.3:

```
@@ -62,4 +62,13 @@
   std::complex<double> topcharge_zero{0., 0.}; // qtilde(k=0), DC component

+  // Spherically-averaged power spectra of the fields from the full 3D grid
+  // (see computeFieldSpectra in measurer.cxx), with the same |k| bins as the
+  // topological charge. field_Sk is NFieldSpectra x NtopchargeBins (row
+  // major): S_c(k) = <|U_c(k)|^2> per radial shell for U = (phi[4], A[3],
+  // V[3]), with U_c(k) = (1/N^3) \sum_x e^{-ikx} U_c(x).
+  static const PetscInt NFieldSpectra = ModelAData::Ndof;
+  std::vector<double> field_Sk;
+  std::vector<double> field_Nshell; // number of modes per shell
+
   // First dimension is NObs, last is spatial index x=0...N
   nvector<PetscScalar, 2> wallX;
@@ -146,11 +155,16 @@
       topcharge_kbins[m] = m * topcharge_dk;
     }
-    // Only rank 0 assembles the full q(x) volume and takes its 3D FFT.
-    if (rank == 0) {
-      fftw3d = std::make_unique<measurer_fft_3d>(N, N, N);
-    }
+    field_Sk.assign(NFieldSpectra * NtopchargeBins, 0.);
+    field_Nshell.assign(NtopchargeBins, 0.);
+    // The 3D FFT machinery is set up on first use (setupSpectra)
   }

- virtual ~Measurer() {}
+ virtual ~Measurer() {
+   VecScatterDestroy(&toZero);
+   VecDestroy(&qseq);
+   VecDestroy(&qnat);
+   VecDestroy(&qglob);
+   DMDestroy(&da1);
+ }

   // Takes the vector, solution,  and computes the walls and their fourier
@@ -178,7 +192,12 @@
   }

+  // Computes field_Sk, the radially binned 3D power spectra of all field
+  // components. Collective, like measure_topcharge.
+  void measure_field_spectra(Vec *solution) { computeFieldSpectra(solution); }
+
   ModelA *getModel() { return model; }
   PetscInt getN() { return N; }
   int getNTopchargeBins() { return NtopchargeBins; }
+  int getNFieldSpectra() { return NFieldSpectra; }
   double getTopchargeDk() { return topcharge_dk; }

@@ -192,4 +211,19 @@
   void computeDerivedObs();
   void computeTopChargeFourier(Vec *solution);
+  void computeFieldSpectra(Vec *solution);
+
+  // 3D spectra of a scalar field q(x). q lives in qglob, a global vector of
+  // the 1-dof DMDA da1 with the same layout as the model's domain.
+  // radialSpectrum brings q to rank 0 in natural order (only a copy for one
+  // rank), takes its FFT there (cuFFT if q is on the device) and bins
+  // |q(k)|^2 into the |k| shells; Sk, Nshell and zero are set on rank 0.
+  PetscErrorCode setupSpectra();
+  PetscErrorCode radialSpectrum(double *Sk, double *Nshell,
+                                std::complex<double> *zero);
+  DM da1 = nullptr;
+  Vec qglob = nullptr;
+  Vec qnat = nullptr;           // natural ordering (more than one rank)
+  Vec qseq = nullptr;           // all of q on rank 0 (more than one rank)
+  VecScatter toZero = nullptr;

   ModelA *model;
@@ -198,5 +232,5 @@
   // FFT engine using the fftw3 library
   std::unique_ptr<measurer_fft> fftw;
-  // 3D FFT engine for the topological charge density (rank 0 only)
+  // 3D FFT engine for the spectra (rank 0 only, made by setupSpectra)
   std::unique_ptr<measurer_fft_3d> fftw3d;
 };
```

1.3 New measurement: 3D power spectra of the fields

Purpose. For each of the 10 field components $U_c = (\phi_0..\phi_3, A_1..A_3, V_1..V_3)$ the code computes the 3D FFT of the full grid and the radially binned power spectrum $S_c(k) = \langle |U_c(k)|^2 \rangle_{\text{shell}}$ with $U_c(k) = N^{-3} \sum_x e^{-ikx} U_c(x)$,

using the same shells and normalization as the topological charge. It is switched on with the new input parameter `saveFrequencyFields` (default -1 = off), which works like `saveFrequencyTopcharge`.

Output (HDF5, measurement file): `fields_Sk` (time × 10 × bins), `fields_time` (time of each row), `fields_kbins` and `fields_Nshell` (written once).

Performance. It reuses the gather, FFT and binning of the topological charge readout: one copy kernel per component, then one FFT and one binning kernel on rank 0. At 256^3 on one GPU this is 10 FFTs of 256^3 per measurement, so it should be called every few steps rather than every step.

Added code (`measurer.cxx`):

```
void Measurer::computeFieldSpectra(Vec *solution) {
  setupSpectra();

  DM &da = model->domain;
  const int nbins = NtopchargeBins;
  std::complex<double> zero; // = the average of the field, not stored
  for (PetscInt c = 0; c < NFieldSpectra; c++) {
#ifdef SFGPU_KOKKOS
    if (is_kokkos_vec(*solution)) {
      field_component_kokkos(da, *solution, c, da1, qglob);
    } else
#endif
    {
      PetscInt xs, ys, zs, xm, ym, zm;
      DMDAGetCorners(da, &xs, &ys, &zs, &xm, &ym, &zm);
      PetscScalar ****u;
      DMDAVecGetArrayDOFRead(da, *solution, &u);
      PetscScalar ***q;
      DMDAVecGetArray(da1, qglob, &q);
      for (PetscInt k = zs; k < zs + zm; k++) {
        for (PetscInt j = ys; j < ys + ym; j++) {
          for (PetscInt i = xs; i < xs + xm; i++) {
            q[k][j][i] = u[k][j][i][c];
          }
        }
      }
      DMDAVecRestoreArray(da1, qglob, &q);
      DMDAVecRestoreArrayDOFRead(da, *solution, &u);
    }
    radialSpectrum(&field_Sk[c * nbins], field_Nshell.data(), &zero);
  }
}
```

Added code input parameter (`ModelA.h`):

```
@@ -120,4 +120,6 @@ struct ModelAHandlerData {
   int saveFrequency = 3;
   int saveFrequencyTopcharge = -1;
+  // Radially binned 3D power spectra of the fields (see Measurer)
+  int saveFrequencyFields = -1;
   int writeFrequency = -1;
   bool compute_energy = false;
@@ -134,4 +136,6 @@ struct ModelAHandlerData {
     saveFrequencyTopcharge =
         params.value("saveFrequencyTopcharge", saveFrequencyTopcharge);
+    saveFrequencyFields =
+        params.value("saveFrequencyFields", saveFrequencyFields);
     writeFrequency = params.value("writeFrequency", writeFrequency);
     compute_energy = params.value("compute_energy", compute_energy);
@@ -147,4 +151,8 @@ struct ModelAHandlerData {
                 outputfiletag.c_str());
     PetscPrintf(PETSC_COMM_WORLD, "saveFrequency = %d\n", saveFrequency);
+    PetscPrintf(PETSC_COMM_WORLD, "saveFrequencyTopcharge = %d\n",
+                saveFrequencyTopcharge);
+    PetscPrintf(PETSC_COMM_WORLD, "saveFrequencyFields = %d\n",
+                saveFrequencyFields);
     PetscPrintf(PETSC_COMM_WORLD, "writeFrequency = %d\n", writeFrequency);
     PetscPrintf(PETSC_COMM_WORLD, "compute_energy = %s\n",
```

Added code call in the time loop (`SuperPions.cxx`):

```
@@ -136,4 +136,17 @@ void run_event(const int &ievent, ModelA *const model, Stepper *const step,
     }

+    // on separate frequency: radially binned 3D power spectra of the fields
+    if (ahandler.saveFrequencyFields > 0 and
+        steps % ahandler.saveFrequencyFields == 0) {
+      PetscLogStagePush(prof.measure_stage);
+      PetscLogEventBegin(measurements, 0, 0, 0, 0);
+      measurer.measure_field_spectra(&model->solution);
+      if (rank == 0) {
+        measurer_output->save_field_spectra();
+      }
```

```
+      PetscLogEventEnd(measurements, 0, 0, 0, 0);
+      PetscLogStagePop();
+    }
+
     // Write the solution to tape if writeFrequency > 0. This is used for
     // plotting of the solution. It is normally not analyzed, or written.
```

Added code HDF5 output (`measurer_output.h`, `measurer_output.cxx`):

```
@@ -21,4 +21,5 @@ public:
   // implement them (e.g. the text backend) fall back to these no-op defaults.
   virtual void save_topcharge(const std::string &what) {}
+  virtual void save_field_spectra(const std::string &what) {}
 };

@@ -53,4 +54,5 @@ public:
   virtual void save(const std::string &what = "") override;
   virtual void save_topcharge(const std::string &what = "") override;
+  virtual void save_field_spectra(const std::string &what = "") override;

 private:
@@ -100,4 +102,13 @@ private:
   std::unique_ptr<ntuple<1>> topcharge_nshell;
   bool topcharge_static_written = false;
+
+  // Radially binned 3D power spectra of the fields (Measurer::field_Sk):
+  // fields_Sk is a time series of NFieldSpectra x nbins rows, fields_time the
+  // time of each row. The |k| bin centers and mode counts are written once.
+  std::unique_ptr<ntuple<2>> fields_sk;
+  std::unique_ptr<ntuple<1>> fields_time;
+  std::unique_ptr<ntuple<1>> fields_kbins;
+  std::unique_ptr<ntuple<1>> fields_nshell;
+  bool fields_static_written = false;
 };
 #endif
```

```
@@ -74,4 +74,13 @@ measurer_output_fasthdf5::measurer_output_fasthdf5(Measurer *in,
   topcharge_zero =
       std::make_unique<ntuple<1>>(NN1, "topcharge_zeromode", file_id);
+
+  // Radially binned 3D power spectra of the fields
+  NN2 = {static_cast<size_t>(measure->getNFieldSpectra()), nbins};
+  fields_sk = std::make_unique<ntuple<2>>(NN2, "fields_Sk", file_id);
+  NN1 = {nbins};
+  fields_kbins = std::make_unique<ntuple<1>>(NN1, "fields_kbins", file_id);
+  fields_nshell = std::make_unique<ntuple<1>>(NN1, "fields_Nshell", file_id);
+  NN1 = {1};
+  fields_time = std::make_unique<ntuple<1>>(NN1, "fields_time", file_id);
 }

@@ -160,4 +169,20 @@ void measurer_output_fasthdf5::save_topcharge(const std::string &what) {
   topcharge_zero->fill();
 }
+
+void measurer_output_fasthdf5::save_field_spectra(const std::string &what) {
+
+  // The bin centers and mode counts are time independent: write them once
+  if (!fields_static_written) {
+    fields_kbins->row = measure->topcharge_kbins;
+    fields_kbins->fill();
+    fields_nshell->row = measure->field_Nshell;
+    fields_nshell->fill();
+    fields_static_written = true;
+  }
+  fields_time->row[0] = measure->getModel()->data.atime.t();
+  fields_time->fill();
+  fields_sk->row = measure->field_Sk;
+  fields_sk->fill();
+}
 #endif
```

1.4 Diffusion stencil (`Stepper.cxx`)

Purpose. `diffusion_stencil_kokkos` is the GPU kernel of each diffusion substep B: the Laplacian of ϕ with the mass term and the normalization $|\phi| = f$, and the Laplacian of the charges A, V. It runs 6 times per step and was 80% of the run time.

What changed. The kernel started from the solution value `phinew(k,j,i,c)` and added the update. The ghosted vector `localU`, which the kernel reads for the neighbours anyway, holds exactly the same value at the centre, because the ghost exchange copied it from the solution just before. The kernel now reads the centre from `localU` and only writes the solution (`DMDAVecGetKokkosOffsetViewDOFWrite`). The arithmetic

is unchanged, so the results are bit-identical.

Performance. The kernel moves two instead of three full field arrays per call (1.34 GB each at 256^3). Nsight Compute: 3.58 ms → 2.54 ms per call (−29%), L1 hit rate 81% → 89%. Step time 57.4 ms → 50.5 ms.

Diff of the kernel (red removed, green added):

```
@@ -1,7 +1,9 @@
 #ifdef SFGPU_KOKKOS
 // Device version of the stencil loop in ModelGExplicitDiffusionStep::step:
 // the same arithmetic in the same order, so with -fmad=false it matches the
-// host loop bit-for-bit.
+// host loop bit-for-bit. The center values are read from localU, which the
+// ghost exchange has just filled from the solution, so the solution itself is
+// only written: one pass over the field less than phinew += ...
 static PetscErrorCode diffusion_stencil_kokkos(
     DM da, Vec localU, Vec solution, const PetscReal (&Hin)[4], PetscReal axx,
     PetscReal ayy, PetscReal azz, PetscReal dtG, PetscReal dtD, PetscReal f) {
@@ -12,7 +14,7 @@
   ConstNodeView phi;
   NodeView phinew;
   PetscCall(DMDAVecGetKokkosOffsetViewDOF(da, localU, &phi));
-  PetscCall(DMDAVecGetKokkosOffsetViewDOF(da, solution, &phinew));
+  PetscCall(DMDAVecGetKokkosOffsetViewDOFWrite(da, solution, &phinew));

   const Kokkos::Array<PetscReal, 4> H = {Hin[0], Hin[1], Hin[2], Hin[3]};
   const PetscReal tiny = std::numeric_limits<double>::min();
@@ -24,7 +26,7 @@
         // phi: nabla^2 phi, then the mass term (H[l] is H * sigma/f)
         PetscReal p[4];
         for (PetscInt l = 0; l < Nphi; l++) {
-          p[l] = phinew(k, j, i, l);
+          p[l] = phi(k, j, i, l);
           p[l] += dtG * (axx * (phi(k, j, i + 1, l) + phi(k, j, i - 1, l)) +
                          ayy * (phi(k, j + 1, i, l) + phi(k, j - 1, i, l)) +
                          azz * (phi(k + 1, j, i, l) + phi(k - 1, j, i, l)) -
@@ -50,7 +52,8 @@

         // Charges A (c = 4..6) then V (c = 7..9): nabla^2 with D
         for (PetscInt c = ModelAData::OffsetA; c < ModelAData::Ndof; c++) {
-          phinew(k, j, i, c) +=
+          phinew(k, j, i, c) =
+              phi(k, j, i, c) +
               dtD * (axx * (phi(k, j, i + 1, c) + phi(k, j, i - 1, c)) +
                      ayy * (phi(k, j + 1, i, c) + phi(k, j - 1, i, c)) +
                      azz * (phi(k + 1, j, i, c) + phi(k - 1, j, i, c)) -
@@ -60,8 +63,8 @@
   // So the DiffusionStencil event times the kernel, not just its launch
   Kokkos::fence();

-  PetscCall(DMDAVecRestoreKokkosOffsetViewDOF(da, solution, &phinew));
+  PetscCall(DMDAVecRestoreKokkosOffsetViewDOFWrite(da, solution, &phinew));
   PetscCall(DMDAVecRestoreKokkosOffsetViewDOF(da, localU, &phi));
   PetscFunctionReturn(PETSC_SUCCESS);
 }
```

1.5 Single-GPU ghost fill (`Stepper.cxx`, `Profiling.h`)

Purpose. Before each substep, `ghost_exchange` copies the solution into a vector with one extra layer of ghost cells (`DMGlobalToLocal`). On the GPU, PETSc does this with an index-list gather (`ScatterAndOp<Insert>`): 4.71 ms per call at 256^3, 7 calls per step, 33 ms of the 57 ms step. Nsight Systems showed that this copy, not the stencil, was the largest cost.

What changed. With one rank and periodic boundaries, every local value is a copy of the global value at the periodically wrapped position. The new kernel `ghost_fill_periodic_kokkos` does this copy directly. Each GPU thread copies one double, with the value index fastest, so neighbouring threads read and write neighbouring memory addresses. `ghost_exchange` uses it when it applies (one rank, periodic boundaries, Kokkos vectors) and PETSc's exchange otherwise. The option `-use_petsc_ghost_exchange` forces the PETSc path.

Performance. 4.71 ms → 1.71 ms per call (2.8×), step time 50.5 ms → 29.7 ms. Final fields are bit-identical. Runs with several ranks still use PETSc's exchange.

Added code dispatch in `ghost_exchange` (`Profiling.h`):

```
@@ -44,4 +44,11 @@ private:
 };

+#ifdef SFGPU_KOKKOS
```

```
+// One kernel instead of DMGlobalToLocal for a single rank with periodic
+// boundaries and Kokkos vectors (Stepper.cxx); *done says whether it applied
+PetscErrorCode ghost_fill_periodic_kokkos(DM da, Vec global, Vec local,
+                                          PetscBool *done);
+#endif
+
 // Fills the ghost cells of local from global, logged as GhostExchange.
 inline PetscErrorCode ghost_exchange(DM da, Vec global, Vec local) {
@@ -49,6 +56,12 @@ inline PetscErrorCode ghost_exchange(DM da, Vec global, Vec local) {
   PetscFunctionBeginUser;
   PetscCall(PetscLogEventBegin(prof.ghost_exchange, 0, 0, 0, 0));
-  PetscCall(DMGlobalToLocalBegin(da, global, INSERT_VALUES, local));
-  PetscCall(DMGlobalToLocalEnd(da, global, INSERT_VALUES, local));
+  PetscBool done = PETSC_FALSE;
+#ifdef SFGPU_KOKKOS
+  PetscCall(ghost_fill_periodic_kokkos(da, global, local, &done));
+#endif
+  if (!done) {
+    PetscCall(DMGlobalToLocalBegin(da, global, INSERT_VALUES, local));
+    PetscCall(DMGlobalToLocalEnd(da, global, INSERT_VALUES, local));
+  }
   PetscCall(PetscLogEventEnd(prof.ghost_exchange, 0, 0, 0, 0));
   PetscFunctionReturn(PETSC_SUCCESS);
```

Added code kernel (`Stepper.cxx`):

```
// Single-rank fast path of ghost_exchange (see Profiling.h). With one rank and
// periodic boundaries every local site, ghosts included, is a copy of the
// global site at its periodically wrapped position, so one kernel fills local
// with one read and one write of the field. PETSc's DMGlobalToLocal does the
// same copy as an index-list gather at a fraction of the memory bandwidth.
// Sets *done to PETSC_FALSE and does nothing when it does not apply (more
// than one rank, other boundaries, host vectors, -use_petsc_ghost_exchange).
PetscErrorCode ghost_fill_periodic_kokkos(DM da, Vec global, Vec local,
                                          PetscBool *done) {
  PetscFunctionBeginUser;
  *done = PETSC_FALSE;
  static PetscBool use_petsc = PETSC_FALSE, checked = PETSC_FALSE;
  if (!checked) {
    PetscCall(PetscOptionsHasName(NULL, NULL, "-use_petsc_ghost_exchange",
                                  &use_petsc));
    checked = PETSC_TRUE;
  }
  PetscMPIInt size;
  PetscCallMPI(MPI_Comm_size(PetscObjectComm((PetscObject)da), &size));
  DMBoundaryType bx, by, bz;
  PetscInt M, N, P;
  PetscCall(DMDAGetInfo(da, NULL, &M, &N, &P, NULL, NULL, NULL, NULL, NULL,
                        &bx, &by, &bz, NULL));
  if (use_petsc || size != 1 || bx != DM_BOUNDARY_PERIODIC ||
      by != DM_BOUNDARY_PERIODIC || bz != DM_BOUNDARY_PERIODIC ||
      !is_kokkos_vec(global) || !is_kokkos_vec(local)) {
    PetscFunctionReturn(PETSC_SUCCESS);
  }

  PetscInt xs, ys, zs, xm, ym, zm;
  PetscCall(DMDAGetGhostCorners(da, &xs, &ys, &zs, &xm, &ym, &zm));
  ConstNodeView g;
  NodeView l;
  PetscCall(DMDAVecGetKokkosOffsetViewDOF(da, global, &g));
  PetscCall(DMDAVecGetKokkosOffsetViewDOFWrite(da, local, &l));
  const int ndof = static_cast<int>(l.extent(3));

  // One thread per value, the (i, c) index of a row fastest: neighbouring
  // threads then copy neighbouring doubles. (One thread per site of ndof
  // values made each warp store touch ndof times more memory sectors.)
  using RowPolicy = Kokkos::MDRangePolicy<
      Kokkos::Rank<3, Kokkos::Iterate::Right, Kokkos::Iterate::Right>,
      Kokkos::IndexType<int>>;
  const int x0 = static_cast<int>(xs), y0 = static_cast<int>(ys),
            z0 = static_cast<int>(zs);
  const int nx = static_cast<int>(M), ny = static_cast<int>(N),
            nz = static_cast<int>(P);
  Kokkos::parallel_for(
      "GhostFill",
      RowPolicy({z0, y0, 0},
                {z0 + static_cast<int>(zm), y0 + static_cast<int>(ym),
                 static_cast<int>(xm) * ndof},
                {1, 1, 256}),
      KOKKOS_LAMBDA(int k, int j, int ic) {
        const int i = x0 + ic / ndof;
        const int c = ic - (i - x0) * ndof;
        // The ghost width is smaller than the box, so one wrap suffices
        const int sk = k < 0 ? k + nz : (k >= nz ? k - nz : k);
        const int sj = j < 0 ? j + ny : (j >= ny ? j - ny : j);
        const int si = i < 0 ? i + nx : (i >= nx ? i - nx : i);
        l(k, j, i, c) = g(sk, sj, si, c);
      });
```

```
  Kokkos::fence();

  PetscCall(DMDAVecRestoreKokkosOffsetViewDOFWrite(da, local, &l));
  PetscCall(DMDAVecRestoreKokkosOffsetViewDOF(da, global, &g));
  *done = PETSC_TRUE;
  PetscFunctionReturn(PETSC_SUCCESS);
}
```

1.6 Job scripts (`super-analysis/run_dcc`)

Purpose. The configurator `sconfigurator_jg.py` builds the input files and calls a submit routine from `sjg_runner.py`, which writes a Slurm script into the run directory and submits it. Before, only Perlmutter (`prlmrun`) was supported.

What changed.

- The configurator detects the platform (Perlmutter or bwUniCluster), and has a switch `gpu` and the number of GPUs per node `ngpus`. It picks the matching executable and submit routine.
- `bwunirun` (CPU nodes): partition `cpu` (`dev_cpu` for `debug=True`), 96 ranks per node, or `ntasks` ranks if given (for example 1 rank for comparisons with a 1-GPU run).
- `bwunirun_gpu` (new): partition `gpu_h100` (`dev_gpu_h100` for `debug=True`), one rank per GPU with 24 cores each, GPU setup script, and the options `-dm_vec_type kokkos -use_gpu_aware_mpi 0`.
- Both start the job environment with a quiet `module purge`, so a conda environment loaded in the submitting shell does not produce error messages in the job.

Diff of the configurator (structural parts only; changes of physics parameters are not shown):

```
@@ -1,4 +1,5 @@
 #!/usr/bin/env python3
 import os
+import functools
 import sjg_runner as qrun
 import sys
@@ -12,5 +13,25 @@ from jg_pymodules import crq
 ############## Global Variables ####################

-EXE_PATH = "/pscratch/sd/j/jannisge/"
+# detect the platform (NERSC sets NERSC_HOST, bwUniCluster 3 nodes are uc3*)
+nersc = os.environ.get("NERSC_HOST") == "perlmutter"
+bwuni = (os.environ.get("SLURM_CLUSTER_NAME") == "uc3"
+         or os.uname().nodename.startswith("uc3"))
+if not (nersc or bwuni):
+    sys.exit("Unknown platform: neither perlmutter nor bwuni detected")
+
+# GPU run (bwuni H100 nodes only): uses the Kokkos/CUDA build in build-gpu
+# and runs one MPI rank per GPU on ngpus (1-4) GPUs per node
+gpu = True
+ngpus = 1
+
+if gpu and nersc:
+    sys.exit("GPU runs are only set up for bwuni")
+
+if nersc:
+    EXE_PATH = "/pscratch/sd/j/jannisge/"
+elif bwuni and gpu:
+    EXE_PATH = "/pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/build-gpu/src/"
+elif bwuni:
+    EXE_PATH = "/pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/build-cpu/src/"
 RESULTS_DIR = "../results/dcc/tests/shock/"
 RESULTS_DIR = "../results/dcc/withnoise/"
@@ -160,5 +180,12 @@ for elem in all_combinations:

     # run / submit the run
-    qrun.prlmrun(program, qin, time=5, debug=False, moreopts=[], dry_run=False, nnodes=16,
+    # adjust function call depending on the platform
+    if nersc:
+        runroutine = qrun.prlmrun
+    elif bwuni and gpu:
+        runroutine = functools.partial(qrun.bwunirun_gpu, ngpus=ngpus)
+    elif bwuni:
+        runroutine = qrun.bwunirun
+    runroutine(program, qin, time=5, debug=True, moreopts=[], dry_run=False, nnodes=1,
                 interactive=interactive, results_dir = RESULTS_DIR)
```

Added code CPU submit routine (`sjg_runner.py`):

```
###############################################################################
############ runs on bwuni ####################################
###############################################################################
```

```
def bwunirun(program, qinput, time=1, append_nodeid=True, debug=False,
             dry_run=True, moreopts=[], seed=None, interactive=False,
             nnodes=1, ntasks=None, results_dir = ""):

    @contextmanager
    def cd(newdir, mkdir=False):
        prevdir = os.getcwd()
        if mkdir and not os.path.exists(newdir):
            os.makedirs(newdir)
        os.chdir(os.path.expanduser(newdir))
        try:
            yield
        finally:
            os.chdir(prevdir)

    # Usage:
    with cd(results_dir + qinput["outputfiletag"], mkdir=True):
    # Everything inside this block happens in the new directory
    # No need for popd; it happens automatically at the end!

        # Append a random 8 digit hex number to the tag labelling the run. This is
        # so that independent runs using the same inputfile, with different seeds,
        # can be run in the same directory. We have to modify the "qinput" structure
        if append_nodeid:
            oldtag = qinput["outputfiletag"]
            runid = str(uuid.uuid4())[:8]
            qinput["outputfiletag"] = qinput["outputfiletag"] + "_{}".format(runid)

        # This is the new tag for the writing below
        tag = qinput["outputfiletag"]

        # Set the seed and write the inputfile to tag.json
        if seed is None:
            qinput["seed"] = random.randint(1, 2000000000)
        else:
            qinput["seed"] = seed

        # Write the current qinput structure to the json file which will
        # be read by the program
        qinputtojson(qinput)

        # Prepare the shell script for running on bwuni
        filenamesh = tag + '.sh'

        fh = open(filenamesh, 'w')

        # all 96 cores per node, unless ntasks (total number of MPI ranks)
        # is given, e.g. ntasks=1 for comparisons with a 1-rank GPU run
        tasks = int(nnodes*96) if ntasks is None else int(ntasks)
        cpuspertask = int(2)
        print("#!/bin/bash", file=fh)
        if debug:
            print("#SBATCH -p dev_cpu", file=fh)
            print("#SBATCH -t 00:30:00", file=fh)
            print("#SBATCH -N {}".format(nnodes), file=fh)
            print("#SBATCH --ntasks={}".format(tasks), file=fh)
            print("#SBATCH --cpus-per-task={}".format(cpuspertask), file=fh)
            print("#SBATCH --output={}".format(tag+".out"), file=fh)
            print("#SBATCH --job-name={}".format(tag), file=fh)
        else:
            print("#SBATCH -p cpu", file=fh)
            print("#SBATCH -t {}".format(int(math.ceil(time*60.))), file=fh)
            print("#SBATCH -N {}".format(nnodes), file=fh)
            print("#SBATCH --ntasks={}".format(tasks), file=fh)
            print("#SBATCH --cpus-per-task={}".format(cpuspertask), file=fh)
            print("#SBATCH --output={}".format(tag+".out"), file=fh)
            print("#SBATCH --job-name={}".format(tag), file=fh)

        # Set up the shell environment (modules + PETSc for the CPU build).
        # Purge the inherited modules quietly first (see bwunirun_gpu: a
        # loaded conda env makes the purge fail noisily).
        env_setup = """
module purge >/dev/null 2>&1
source /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/platforms/bwuni3/setupbwuni3.sh
"""
        print(env_setup, file=fh)
        print("#run the application:", file=fh)
        print('date  "+%%x %%T" > %s_time.out' %
              (qinput["outputfiletag"]), file=fh)
        # Write the command that actually runds the program
        print("srun --mpi=pmix -n %d --cpu_bind=cores -c %d %s -input %s " %
              (tasks, cpuspertask, program, qinput["outputfiletag"]+'.json'), end=' ', file=fh)

        # This additional options are  added to the srun command
        print("-log_view", end=' ', file=fh)
        for opt in moreopts:
            print(opt, end=' ', file=fh)
        print(file=fh)
```

```
        # # Do any post processing of the run
        print('date  "+%%x %%T" >> %s_time.out' %
              (qinput["outputfiletag"]), file=fh)
        fh.close()

        # Submit the shell script to the batch system, executing
        #
        # sbatch tag.sh
        if not dry_run:
            if interactive:
                subprocess.run(['bash', filenamesh])
            else:
                subprocess.run(['sbatch', filenamesh])

        # There was a side effect that the outputfiletag got modified
        # This should be undone for transparency
        if append_nodeid:
            qinput["outputfiletag"] = oldtag
    # return to the root directory
```

Added code GPU submit routine (`sjg_runner.py`):

```
###############################################################################
############ GPU runs on bwuni (H100 nodes) ###################################
###############################################################################

def bwunirun_gpu(program, qinput, time=1, append_nodeid=True, debug=False,
                 dry_run=True, moreopts=[], seed=None, interactive=False,
                 nnodes=1, ngpus=4, results_dir = ""):

    @contextmanager
    def cd(newdir, mkdir=False):
        prevdir = os.getcwd()
        if mkdir and not os.path.exists(newdir):
            os.makedirs(newdir)
        os.chdir(os.path.expanduser(newdir))
        try:
            yield
        finally:
            os.chdir(prevdir)

    with cd(results_dir + qinput["outputfiletag"], mkdir=True):

        # Append a random 8 digit hex number to the tag labelling the run
        # (see bwunirun)
        if append_nodeid:
            oldtag = qinput["outputfiletag"]
            runid = str(uuid.uuid4())[:8]
            qinput["outputfiletag"] = qinput["outputfiletag"] + "_{}".format(runid)

        # This is the new tag for the writing below
        tag = qinput["outputfiletag"]

        # Set the seed and write the inputfile to tag.json
        if seed is None:
            qinput["seed"] = random.randint(1, 2000000000)
        else:
            qinput["seed"] = seed

        qinputtojson(qinput)

        # Prepare the shell script for running on the bwuni H100 nodes
        filenamesh = tag + '.sh'

        fh = open(filenamesh, 'w')

        # 4 H100s and 96 cores per node; one MPI rank per GPU, so ngpus
        # (1-4) GPUs per node and 96/4 = 24 cores per rank
        tasks = int(nnodes*ngpus)
        cpuspertask = int(96/4)
        print("#!/bin/bash", file=fh)
        if debug:
            print("#SBATCH -p dev_gpu_h100", file=fh)
            print("#SBATCH -t 00:30:00", file=fh)
        else:
            print("#SBATCH -p gpu_h100", file=fh)
            print("#SBATCH -t {}".format(int(math.ceil(time*60.))), file=fh)
        print("#SBATCH -N {}".format(nnodes), file=fh)
        print("#SBATCH --ntasks-per-node={}".format(ngpus), file=fh)
        print("#SBATCH --cpus-per-task={}".format(cpuspertask), file=fh)
        print("#SBATCH --gres=gpu:{}".format(ngpus), file=fh)
        print("#SBATCH --output={}".format(tag+".out"), file=fh)
        print("#SBATCH --job-name={}".format(tag), file=fh)

        # Set up the shell environment (modules + CUDA/Kokkos PETSc).
        # The job inherits the submitting shell's modules; purging a loaded
        # conda env runs 'conda deactivate' with the env's python, which fails
```

```
        # noisily, so purge quietly first. OMP_* binds Kokkos' host threads.
        env_setup = """
module purge >/dev/null 2>&1
source /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/platforms/bwuni3/setupbwuni3-gpu.sh
export OMP_PROC_BIND=spread
export OMP_PLACES=threads
"""
        print(env_setup, file=fh)
        print("#run the application:", file=fh)
        print('date  "+%%x %%T" > %s_time.out' %
              (qinput["outputfiletag"]), file=fh)
        # Every rank sees all GPUs of its node and PETSc gives each rank its
        # own GPU. -dm_vec_type kokkos puts the fields on the GPU. The
        # OpenMPI modules are not CUDA-aware, so PETSc must stage the ghost
        # exchange through host memory (-use_gpu_aware_mpi 0).
        print("srun --mpi=pmix -n %d --cpu_bind=cores -c %d %s -input %s " %
              (tasks, cpuspertask, program, qinput["outputfiletag"]+'.json'), end=' ', file=fh)
        print("-dm_vec_type kokkos -use_gpu_aware_mpi 0", end=' ', file=fh)

        # This additional options are  added to the srun command
        print("-log_view", end=' ', file=fh)
        for opt in moreopts:
            print(opt, end=' ', file=fh)
        print(file=fh)

        print('date  "+%%x %%T" >> %s_time.out' %
              (qinput["outputfiletag"]), file=fh)
        fh.close()

        # Submit the shell script to the batch system (sbatch tag.sh)
        if not dry_run:
            if interactive:
                subprocess.run(['bash', filenamesh])
            else:
                subprocess.run(['sbatch', filenamesh])

        # Undo the side effect on outputfiletag
        if append_nodeid:
            qinput["outputfiletag"] = oldtag
```

1.7 Validation

Test	Result
CPU vs GPU, $N = 64$, same seed, 1 rank each, 40 steps	initial fields bit-identical; later max $\lvert\Delta\rvert$ 10^{-15}–$4 \cdot 10^{-14}$ (a few ulp), not growing
Spectra, $N = 64$, CPU and GPU with 1 and 2 ranks (2 ranks = 2 GPUs)	GPU vs CPU $\leq 10^{-13}$ relative for $k > 0$; each run vs numpy recomputation from snapshots $\leq 10^{-14}$
Stencil change and ghost fill, 256^3, 20 steps	final fields bit-identical to the starting code
Ensemble $N = 256$, 5 GPU events vs 2 CPU events	$M^2(t)$ agrees for $t \gtrsim 40$; at early times the GPU mean is about 25% higher, with error bands that barely overlap (probably statistics with 5 and 2 events)

Two details: the GPU spectra sum with atomic additions, so repeated GPU runs differ in the last digits (about $5 \cdot 10^{-15}$). With `zero_charges` the charge spectra at $k = 0$ are 0 on the CPU and about 10^{-37} on the GPU (rounding of a zero sum). The notebook `~/super_jupyter/Comp_test.ipynb` contains the plots of these comparisons.

2 Running the code

2.1 The run modes

	CPU	1 GPU	several GPUs, 1 node	several nodes
Build directory	`build-cpu`	`build-gpu`	`build-gpu`	`build-gpu`
Setup script	`setupbwuni3.sh`	`setupbwuni3-gpu.sh`	`setupbwuni3-gpu.sh`	`setupbwuni3-gpu.sh`
Partition	`cpu`	`gpu_h100`	`gpu_h100`	`gpu_h100`
MPI ranks	96 per node	1	2–4 (one per GPU)	4 per node
Ghost exchange	PETSc (host)	GPU ghost fill	PETSc, staged via host	PETSc, staged via host
Tested	yes	yes	yes, 2 GPUs	no

Which to use. One H100 (94 GB) holds up to about 768^3 (tight) and runs a 256^3 step about 9× faster than a 128-core Perlmutter CPU node. For ensembles of events at $N \leq 512$, many independent 1-GPU jobs give the most events per hour. Several GPUs are needed only for lattices that do not fit on one GPU (about 1024^3 needs 4 GPUs).

Random initial conditions depend on the number of ranks: every rank seeds its own random generator from the base seed. Runs with the same seed only start from the same fields if they use the same number of ranks.

2.2 One-time setup: PETSc

Two PETSc 3.25.5 installations (64-bit indices, HDF5 1.14.6 built by PETSc) are installed in `$HOME/opt`:

- CPU: `petsc-3.25.5-gnu14.2-int64`, built with `platforms/bwuni3/build-petsc-cpu.sh`.
- GPU: `petsc-3.25.5-gnu14.2-cuda-int64` (CUDA 12.8, `sm_90`, Kokkos and Kokkos-Kernels 5.1.0), built with `platforms/bwuni3/build-petsc-gpu.sh`.

Both scripts are Slurm jobs (`sbatch build-petsc-*.sh` from `platforms/bwuni3`); the package tarballs are in `petsc-build/pkgs`, so the compute node needs no network. Both use the modules of `platforms/bwuni3/modules.sh`:

```
module purge
module load compiler/gnu/14.2
module load mpi/openmpi/5.0-gnu-14.2
module load numlib/fftw/3.3.10_gnu_14.2_openmpi_5.0
```

GPU configure options:

```
./configure --prefix=$HOME/opt/petsc-3.25.5-gnu14.2-cuda-int64 \
  --with-cc=mpicc --with-cxx=mpicxx --with-fc=0 \
  --with-64-bit-indices --with-debugging=0 \
  --with-cuda --with-cuda-dir=$CUDA_HOME --with-cuda-arch=90 --with-openmp \
  --download-kokkos=$PKGS/kokkos-5.1.0.tar.gz \
  --download-kokkos-kernels=$PKGS/kokkos-kernels-5.1.0.tar.gz \
  --download-f2cblaslapack=$PKGS/f2cblaslapack-3.8.0.q2.tar.gz \
  --download-hdf5=$PKGS/hdf5-1.14.6.tar.gz \
  COPTFLAGS=-O3 CXXOPTFLAGS=-O3 CUDAOPTFLAGS=-O3
```

The CPU build uses `-with-cuda=0`, no Kokkos, and `-O3 -march=native`.

2.3 Compiling SuperPions

CPU build

```
cd /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu
source platforms/bwuni3/setupbwuni3.sh
cmake -B build-cpu -DCMAKE_BUILD_TYPE=Release
cmake --build build-cpu -j
# executable: build-cpu/src/SuperPions.exe
```

GPU build

```
cd /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu
source platforms/bwuni3/setupbwuni3-gpu.sh
cmake -B build-gpu -DSFGPU_KOKKOS=ON \
  -DCMAKE_EXE_LINKER_FLAGS="-L$CUDA_HOME/lib64/stubs -Wl,--no-as-needed -lnvidia-ml -Wl,--as-needed"
cmake --build build-gpu -j
# executables: build-gpu/src/SuperPions.exe, build-gpu/tests/kokkos_smoke.exe
```

The linker flags are needed only when compiling on a node without an NVIDIA driver, such as the login nodes: the PETSc library references the driver library `libnvidia-ml`, and the flag links against the stub library of the CUDA toolkit. At run time on a GPU node the real driver library is loaded. The login node has one core, so a full build takes a few minutes.

2.4 Starting jobs

With the configurator (recommended)

Use the `super` conda environment (the configurator imports `h5py`), set the parameters and the run mode in `sconfigurator_jg.py`, and run it from `run_dcc`:

```
module load devel/miniforge/25.3.1-python-3.12 && conda activate super   # alias: superenv
cd /pfs/work9/workspace/scratch/hd_pn277-super/super-analysis/run_dcc
python sconfigurator_jg.py
```

Mode	Settings in `sconfigurator_jg.py`	Resulting job
CPU	`gpu = False`, `nnodes=n`	`cpu`, $96n$ ranks
1 GPU	`gpu = True`, `ngpus = 1`	`gpu_h100`, 1 rank, 1 GPU
4 GPUs, 1 node	`gpu = True`, `ngpus = 4`, `nnodes=1`	`gpu_h100`, 4 ranks, 4 GPUs
several nodes	`gpu = True`, `ngpus = 4`, `nnodes=n`	`gpu_h100`, $4n$ ranks (untested)

In the call `runroutine(...)` at the end, `debug=True` uses the development queues (`dev_cpu`, `dev_gpu_h100`, 30 minutes), `debug=False` the normal queues with `time` hours; `dry_run=True` only writes the input file and job script.

By hand: CPU job

```
#!/bin/bash
#SBATCH -p cpu
#SBATCH -t 120
#SBATCH -N 1
#SBATCH --ntasks=96
#SBATCH --cpus-per-task=2
module purge >/dev/null 2>&1
source /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/platforms/bwuni3/setupbwuni3.sh
srun --mpi=pmix -n 96 --cpu_bind=cores -c 2 \
  /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/build-cpu/src/SuperPions.exe \
  -input run.json -log_view
```

By hand: one GPU

```
#!/bin/bash
#SBATCH -p gpu_h100
#SBATCH -t 120
#SBATCH -N 1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=24
#SBATCH --gres=gpu:1
module purge >/dev/null 2>&1
source /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/platforms/bwuni3/setupbwuni3-gpu.sh
export OMP_PROC_BIND=spread OMP_PLACES=threads
srun --mpi=pmix -n 1 --cpu_bind=cores -c 24 \
  /pfs/work9/workspace/scratch/hd_pn277-super/superfluid-gpu/build-gpu/src/SuperPions.exe \
  -input run.json -dm_vec_type kokkos -use_gpu_aware_mpi 0 -log_view
```

By hand: four GPUs on one node

Change the resource lines and the rank count; everything else stays as for one GPU. Every rank sees all GPUs of the node and PETSc gives each rank its own GPU.

```
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=24
#SBATCH --gres=gpu:4
srun --mpi=pmix -n 4 --cpu_bind=cores -c 24 .../build-gpu/src/SuperPions.exe \
  -input run.json -dm_vec_type kokkos -use_gpu_aware_mpi 0 -log_view
```

For several nodes use `-N n` and `-n 4n`. This has not been tested.

2.5 Options and input parameters

Command-line option	Meaning
`-input run.json`	input file (written by the configurator)
`-dm_vec_type kokkos`	keep the fields on the GPU; required for GPU runs
`-use_gpu_aware_mpi 0`	required with the Open MPI modules, which are not CUDA-aware: PETSc copies GPU data through host memory for MPI
`-use_petsc_ghost_exchange`	force PETSc's ghost exchange instead of the single-GPU ghost fill (for comparisons)
`-log_view`	timing table at the end of the output
JSON input parameter	
`saveFrequency`	slice averages and wall spectra every n steps
`saveFrequencyTopcharge`	topological charge spectrum every n steps (-1: off)
`saveFrequencyFields`	field spectra `fields_Sk` every n steps (-1: off; new)
`writeFrequency`	full field snapshots `_t_<time>_save.h5` every n steps (-1: off)

2.6 Notes for multi-GPU runs

- **CUDA-aware MPI.** The site modules `mpi/openmpi/5.0*` are not CUDA-aware (the CUDA plugin of Open MPI is not installed), although the comment in `build-petsc-gpu.sh` says otherwise. With `-use_gpu_aware_mpi 0` every ghost exchange between GPUs goes through host memory. The NVIDIA HPC SDK contains a CUDA-aware Open MPI 4.1.7 (HPC-X 2.21, `/software/bwhpc/common/toolkit/nvidia_hpc_sdk/25.1/Linux_x86_64/25.1/comm_libs/12.6/hpcx`); using it requires rebuilding PETSc against it.
- **Ghost exchange.** The fast GPU ghost fill is used only with one rank; with several ranks PETSc's exchange is used.
- **Memory on rank 0.** For the spectra, rank 0 holds the full grid of one scalar plus its FFT on its GPU (about 17 GB plus cuFFT workspace at 1024^3).
- **Node hardware** (bwHPC wiki): `gpu_h100` has 19 nodes with 4×H100 (94 GB), 96 cores, 768 GB RAM and 4×NDR200 InfiniBand. Whether the 4 GPUs are NVLink-connected can be checked with `nvidia-smi topo -m` on a node.